Jury questions and answers

English version of ../qa-jury.md. A preparation for the defence. Part I covers product questions, part II technical ones. The answers are short and reference repository files; places where a decision has not been made yet are marked explicitly.

Part I. Product questions

1. What problem does the system solve?

It brings interactions with universities, statuses, tasks, comments, documents and management reports into a single working process.

2. Who is the main user?

The KAM who runs a portfolio of universities. An executive uses the reports, and an administrator manages reference data and the workflow.

3. Why Kanban?

Kanban shows bottlenecks quickly: the number of cards at every stage, overdue actions and the next step for a particular university.

4. Can the 14 stages be changed?

Yes. The administrator can add, rename, reorder, enable, disable or delete a stage with a transfer of interactions.

5. What happens if a stage with cards is disabled?

The backend rejects the dangerous operation. Active interactions have to be moved first, or the stage deleted with an explicitly chosen target stage.

6. How is access restricted?

JWT authentication and RBAC are checked on the backend. Administrative operations are unavailable to a role without the corresponding permission.

7. Where is the data stored?

Two contours are supported today. The main one is Yandex Cloud: a VM, nginx, PostgreSQL and Object Storage, with the data placed in Russia. The alternative is Render (API) + Vercel (frontend) + Supabase (PostgreSQL), used as the public demonstration stand. The code is the same; the difference is in the environment variables.

8. Why was Yandex Cloud chosen as the main contour?

The requirements of 152-FZ on the localisation of databases of Russian citizens and FSTEC order No. 117 assume placement in the Russian jurisdiction. Yandex Cloud provides certified infrastructure, and the whole stack is already containerised, so the move is done by changing environment variables without touching the code.

9. How is 152-FZ ensured?

The product has an audit, RBAC, HTTPS/CORS, a privacy policy and cookie consent. Data localisation requires a separate confirmation of the chosen production contour.

10. What data counts as personal?

The contacts of users and university representatives, credentials and technical identifiers. They must not be placed in test data or public screenshots.

11. How does export work?

The frontend performs an authorized Axios request as a Blob, the backend returns `Content-Disposition`, and the browser saves the XLSX, XLS or PDF.

12. How is performance verified?

The repository contains a Locust scenario: login, reports, Kanban and reference data. The target criteria are 50 users, 60 seconds, p95 below 1000 ms.

13. Are there actual load results already?

Yes, a local run was performed: 50 users / 60 seconds, 1613 requests, 0 errors, aggregate p95 93 ms; for `/api/reports` p95 7 ms. The reports are in `tests/load/report.html` and `tests/load/report-10-reports.html`. These are local stand figures, not a confirmed SLA on the customer's VM — that needs re-verification on the target contour.

14. What happens when the backend is unavailable?

The UI shows a loading state or an error without masking the absence of data. Once the connection is restored, TanStack Query repeats the request.

15. How does the system scale?

The frontend is static, the backend can run as several instances, and PostgreSQL scales with the means of the chosen managed contour.

16. Why FastAPI?

It provides typed schemas, an asynchronous stack and automatic OpenAPI documentation with a small amount of infrastructure code.

17. How is history loss prevented?

Modifying operations go through the audit middleware, and workflow operations preserve the link between the card, the stage and the user.

18. How is data restored?

For Yandex Cloud a daily `pg_dump` is prepared, keeping the last seven copies. A restore must be verified on a test database regularly.

19. What has been done for mobile devices?

Metrics reflow into a grid, forms become vertical, tables get local horizontal scrolling, and the footer respects the safe area. The cookie banner on a phone is limited to 55vh, the buttons stack vertically, and the settings modal fits the screen.

20. What is the next step after the defence?

Run a real load test, close the browser smoke test, confirm the data-storage contour and agree on migrating production to a Russian cloud.

Part II. Technical questions

1. How does the two-way integration with LMS and CMS work?

The exchange contract is described by the endpoint `GET /api/integration/schema`, package acceptance by `POST /api/integration/import`, a dry run without writing by `POST /api/integration/preview`, and the outbound export by `GET /api/integration/outbound` (see `backend/api/integration.py`). A repeated import matches the card by contract number and updates it instead of creating a duplicate; unknown fields do not break parsing but are returned in `warnings`. Pulling from an external system (`GET /api/integration/pull/{source}`) still reads a stub, because the LMS/CMS contract was not provided by the customer — moving to a live source comes down to replacing the data source.

2. Why don't you version the workflow?

Versioning was not required by the specification, and a stage change applies to all active cards at once — this is a conscious choice in favour of simplicity for the administrator. Stages are split by the `scope` field into two sets: B2B (the 14 stages of the IT School) and B2C (a simplified funnel), see `backend/models/entities.py`. The change history is partly covered by the audit: a transition is recorded in `action_logs` with the old and the new value. If versions are needed, the next step is a separate workflow version table on top of the current schema.

3. How is the audit log implemented?

Records are created by the middleware `backend/middleware/audit.py` for every successful modifying request (`POST`, `PUT`, `PATCH`, `DELETE` with a status below 400); `GET/OPTIONS/HEAD` and the login and health endpoints are skipped. `action_logs` stores the user, the action (CREATE/UPDATE/DELETE), the entity type and id, the old and the new value and the IP address. The journal is read by the administrator through `GET /api/audit` with filters and a CSV export (`backend/api/audit.py`) — the data can not only be accumulated but also shown to an auditor.

4. What about 152-FZ, where is personal data stored?

Personal data means the contacts of university representatives, the users' credentials and technical identifiers; they do not reach test data or public screenshots. The main contour is Yandex Cloud: a VM, nginx, PostgreSQL and Object Storage in Russia, which covers the localisation of Russian citizens' data (`docs/architecture/deployment-yandex-cloud.md`). The technical measures are RBAC, audit, HTTPS/CORS and security headers (`backend/middleware/security_headers.py`). The localisation of a specific production contour requires a separate confirmation from the customer; this is part of the next-step plan.

5. How does RBAC work at the request level?

The role comes from the JWT, and the access level is checked by FastAPI dependencies: `get_current_user`, `require_manager_or_admin`, `require_admin`. Point operations by id go through `backend/api/access.py`: a KAM (the `user` role) sees only their own cards, the check is done by `assigned_kam_id`, and someone else's id gives 403 rather than opening the card. The logic lives in one module so that it applies identically to interactions and to files.

6. How do you handle 50 concurrent users?

The scenario is `tests/load/locustfile.py`: login, reports, Kanban and reference data. The run was performed locally, the reports are in `tests/load/`: **50 users / 60 seconds, 1613 requests, 0 errors, aggregate p95 93 ms**. A separate `ReportsUser` profile produced 322 requests, 0 errors, with p95 7 ms for `/api/reports`. The load is served by the asynchronous stack (FastAPI + SQLAlchemy async), a 30-second report aggregate cache (`backend/services/report_cache.py`) and indexes on the filtered card fields. A production SLA has not been confirmed separately — these are local run figures, not a guarantee for the customer's VM.

7. What about leaks, was a profiler run?

A memory profiler was not run, and we will not present its results — that is the honest position. What already limits leaks: the request body for the audit is truncated, the database session state lives inside the request (`SessionLocal` as a context manager), there is no global mutable state per request, and the report cache has a TTL. The next step is a `tracemalloc`/`py-spy` run under load and recording a baseline in the report.

8. How is the XLS/XLSX import built, where is the dialogue-based mapping?

Catalogue import is `backend/services/excel_import.py`: `openpyxl` for XLSX, `xlrd` for XLS, and the structure is validated with `jsonschema`. Dialogue-based column mapping exists in the UI (`frontend/src/pages/DirectoryPage.tsx`): after a file is chosen, a "Field mapping" modal opens, the user sees up to five previewed rows and maps the columns, and the chosen schema is saved in `localStorage` per reference-data type and reused on the next import. Validation happens on the backend through `preview`, and mandatory fields cannot be left unmapped. Errors are returned per row (`ValidationIssue`: row, field, problem, level — an error blocks the row, a warning does not), and a report about them is exported to XLSX (`backend/services/import_report.py`).

9. How were write operations tested?

Modifying scenarios are covered by integration tests that go through the API into a real database: `test_audit_log.py`, `test_interaction_access.py`, `test_user_delete.py`, `test_workflow_scope.py`, `test_integration.py`, `test_excel_import.py`, `test_json_export.py`. In total the suite has 200 passing tests, plus 22 Keycloak tests separately. No mocks are used: real write paths, integrity constraints and audit recording are verified.

10. What about scaling to 300+?

The backend has no state of its own, so it scales with replicas behind a balancer; sessions, files and the cache are moved out to PostgreSQL, S3 and Redis/KeyDB (`docs/architecture/ARCHITECTURE.md`, the "Scalability" section). What is already in the code: a connection pool, indexes for board and report filtering, and a 30-second aggregate cache. As the load grows, the next step is PostgreSQL read replicas and moving reports to a separate pool.

11. Why KeyDB and not Redis?

The contour currently uses Redis 7, and KeyDB is a multi-threaded fork of Redis with the same protocol, recommended by the customer as lighter (`docs/STACK.md`). The move requires no code changes: the address is set by the `redis_url` environment variable, and `backend/services/report_cache.py` works through an ordinary Redis client. That is a replacement of the operational layer, not a rewrite of the logic.

12. How do you handle deleting a status that has active tasks?

Deleting a stage is `DELETE /api/stages/{stage_id}` with an optional `target_stage_id` (`backend/api/stages.py`). The database has `ON DELETE RESTRICT`, so an active stage cannot be deleted while it holds

cards: without `target_stage_id` a 409 is returned with the number of interactions, and with it the cards are moved to the target stage and only then is the stage deleted. This protects against losing tasks while cleaning up the funnel.

13. How is the JSON export implemented?

`GET /api/reports/json` (`backend/api/reports.py`) returns a reproducible export: `generated_at`, `total`, a column description from the shared config `config/report_columns.json`, a `storage` block (s3 or local and the bucket name), the applied filters and the interactions themselves with file keys. Following Krylov's Q&A requirement, the export contains not only card data but also the link keys, so the receiving side can match objects and download attachments. The columns and filters are the same as in the interface and in the PDF/XLS exports.

14. What about the LLM — how do you justify using it?

The LLM is used for a single optional feature — the card summary (FT-6), see `backend/services/gigachat.py`. Without the `GIGACHAT_CREDENTIALS` key the feature is off and returns 503, and the interface hides the button; external service errors never turn into a 500. Only comment texts and the names of tasks, the university and the product go into the prompt — personal data is excluded, and the demo summary that does not call the model is explicitly marked with a separate value of the "Model" field.

15. How do you deploy — Docker, Kubernetes?

The main way is Docker and Docker Compose: the base `docker-compose.yml` and the production stack `infra/yandex-cloud/docker-compose.prod.yml` with nginx (`docs/DEPLOYMENT.md`). Kubernetes is not used in the current contour; it is named as the path for horizontal scaling once a single VM is no longer enough. That choice is justified by the load size and the timeline: one VM with containers is simpler to maintain, and moving to an orchestrator needs no code changes.

16. What about data integrity under a concurrent update?

Transactions go through the SQLAlchemy session, and critical invariants are checked on the server: the limit of active interactions per university `_ensure_parallel_limit` (`backend/api/interactions.py`), a match between the card's `scope` and the stage's (409), and the uniqueness of stages by `(scope, code)` and `(scope, order)`. There is no explicit optimistic locking (no version column, no `SELECT FOR UPDATE`) yet — when one card is edited concurrently, "last write wins" applies, and that is a documented next step. The proposed solution is a `version` column or row locking on conflicting operations.

17. How do you rank the programmes?

There is no separate programme ranking algorithm in the system, and we will not pretend there is. The reports provide aggregates and filters by stage, university, product and direction, and sorting is done by the export columns (`config/report_columns.json`). If the customer needs a priority list, that is formulated as a separate task with explicit criteria weights rather than implied by the current logic.

18. Why Python and not Kotlin?

The specification itself suggests Python and PostgreSQL, and FastAPI closes the task with no gain from changing the language: asynchronous I/O for a load of 300+, typed schemas and a shared type language with Pydantic, automatic OpenAPI documentation (`docs/architecture/ARCHITECTURE.md`). Kotlin would demand a separate team and a rewrite with no advantage for CRM tasks. The team knows Python, so the speed of development and maintenance is higher.

19. What has been done for mobile devices?

The interface is responsive: metrics reflow into a grid, forms become vertical, tables get local horizontal scrolling, and the footer respects the safe area. The cookie banner on a phone is limited to 55vh with internal scrolling, buttons stack full width, and the settings modal fits the screen and does not cover its own footer. There is no separate native application — the KAM scenario is covered by the browser on a phone. If required, the next step is a PWA shell on top of the current SPA.

20. What about the documentation — is it built into the platform?

Yes, the documentation is built in: the "Help" tab (`/help`) serves Markdown from the static mirror `frontend/public/docs/`, assembled by `scripts/sync_public_docs.py`. Next to it sit the single PDF (`docs/rtk-crm-documentation.pdf`), the separate user and administrator guides and the architecture diagrams. The mirror is checked by a drift test (`tests/test_docs_mirror.py`), so "Help" does not diverge from the sources.